Experiment No.9
Convert an Infix expression to Postfix expression using stack. Implement a program to Convert an Infix expression to Postfix expression using stack
Date of Performance:
Date of Submission:

Experiment No. 9 : Convert an Infix expression to Postfix expression using stack.

Aim: Implement a program to Convert an Infix expression to Postfix expression using stack.

Objectives:

1) Learn the difference between infix and postfix expressions, and how postfix notation eliminates the need for parentheses and operator precedence rules.
2) Develop a function using a stack to convert an infix expression to a postfix expression by handling operators, operands, and parentheses.
3) Apply the algorithm to convert various infix expressions into postfix and ensure the correctness of the result through validation.

Theory:

Infix to Postfix conversion is the process of converting an expression written in infix notation (where operators are placed between operands) into postfix notation (where operators follow their operands). This conversion is useful because postfix expressions eliminate the need for parentheses and operator precedence rules, making the expression easier to evaluate programmatically.

Basic Operations in Infix to Postfix Conversion:

1. Scan the Infix Expression: Start from the leftmost character and move towards the right of the infix expression.
2. Process Operands: When an operand (number or variable) is encountered, directly add it to the postfix expression.
3. Process Operators: When an operator is encountered, pop operators from the stack to the postfix expression until an operator with lower precedence is at the top of the stack, then push the current operator onto the stack.
4. Process Parentheses:
    - Left Parenthesis '(': Push it onto the stack as it indicates the start of a sub-expression.

    - Right Parenthesis ')': Pop operators from the stack to the postfix expression until a left parenthesis is encountered. Discard the left parenthesis.
5. Pop Remaining Operators: After scanning the entire infix expression, pop any remaining operators from the stack to the postfix expression.

Steps for Infix to Postfix Conversion:

1. Initialize an Empty Stack: The stack will be used to store operators and parentheses during the conversion process.
2. Read the Infix Expression Left to Right: For each character:
    - If it's an operand, add it directly to the postfix expression.
    - If it's an operator, pop operators from the stack to the postfix expression until the operator at the top of the stack has lower precedence, then push the current operator onto the stack.
    - If it's a left parenthesis '(', push it onto the stack.
    - If it's a right parenthesis ')', pop all operators from the stack to the postfix expression until a left parenthesis is encountered, then discard the left parenthesis.
3. Pop Remaining Operators: Once the infix expression is fully scanned, pop any remaining operators from the stack and add them to the postfix expression.

Example of Infix to Postfix Conversion:

- Infix Expression: (A + B) * C
- Postfix Expression: A B + C *

This process ensures that the expression is converted into a form that can be easily evaluated without worrying about operator precedence or parentheses.

Algorithm:

```
function infixToPostfix(expression):

   stack = []

   postfix = []

   for each character in expression:

      if character is an operand (A-Z, 0-9):

         add character to postfix

      else if character is '(':
```

```
        push character to stack
    else if character is ')':
        while stack is not empty and top of stack is not '(':
        pop from stack and add to postfix
        pop '(' from stack
    else if character is an operator (+, -, *, /):
            while stack is not empty and precedence of top of stack >= precedence of current operator:
        pop from stack and add to postfix
        push current operator to stack
  while stack is not empty:
    pop from stack and add to postfix
  return postfix as a string
```

Code and Output:

Conclusion: Students will conclude their understanding of Implementation of converting Infix expression to Postfix expression using Stack.